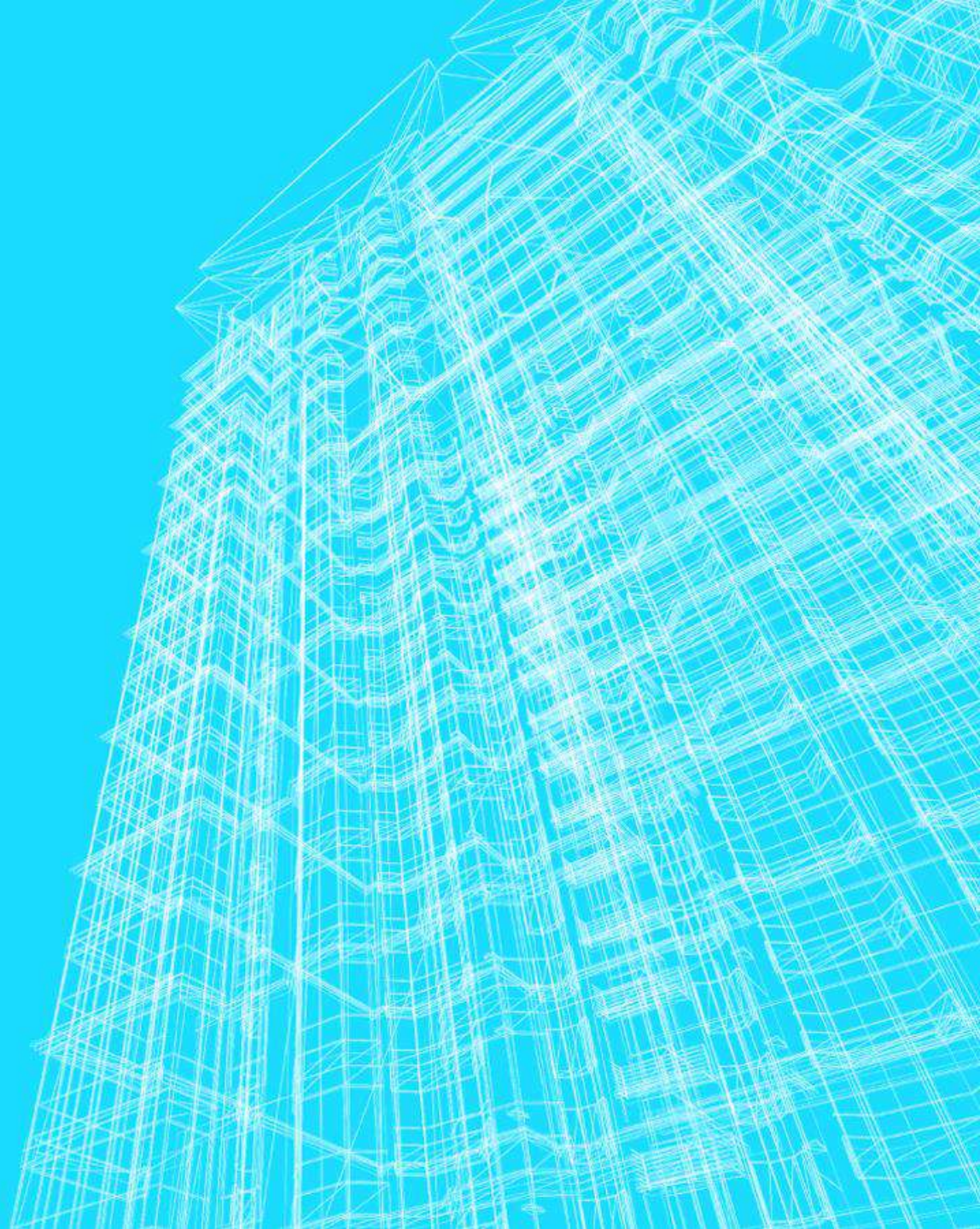


BACKTRACKING (RETROCESSO)

João Pedro Oliveira Batisteli



BACKTRACKING

- Técnica de projeto de algoritmos também conhecida como retrocesso ou tentativa e erro.
- Refinamento da busca exaustiva/força bruta:
 - Elimina algumas soluções sem examinar.

BACKTRACKING

- **Tentativa e Erro Estruturada:** O algoritmo tenta construir uma solução adicionando uma peça de cada vez (como colocar um item na mochila ou um número no Sudoku).
- **O Retrocesso (Backtrack):** O grande diferencial é a capacidade de "**desfazer**". Se a peça que acabamos de colocar torna a solução impossível, o algoritmo desfaz essa última ação (**retrocede**) e tenta a próxima peça disponível. Ele tem "memória" de onde parou.

BACKTRACKING

- **O Problema da Força Bruta:** Como vimos na aula anterior, a força bruta gera todas as respostas possíveis até o final e só depois verifica se elas servem. Isso gera um **esforço gigantesco**.
- **O Refinamento (A Poda / Pruning):** O backtracking verifica as regras do problema durante a construção da resposta.

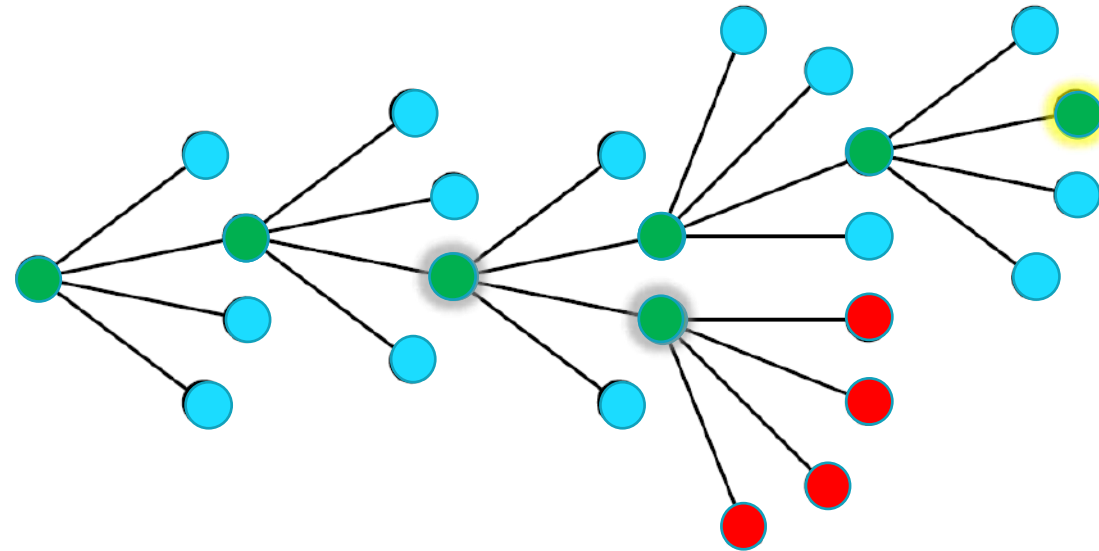
BACKTRACKING

Exemplo Prático (A Mochila):

1. Imagine que a mochila suporta 20kg.
2. Se no primeiro passo o algoritmo escolhe um item que pesa 25kg, a mochila já estourou.
3. O backtracking percebe isso na hora e não perde tempo gerando o resto das combinações que incluem aquele item.
4. Ele corta (poda) esse galho inteiro da árvore de possibilidades, eliminando milhares de combinações ruins sem nunca precisar testá-las.

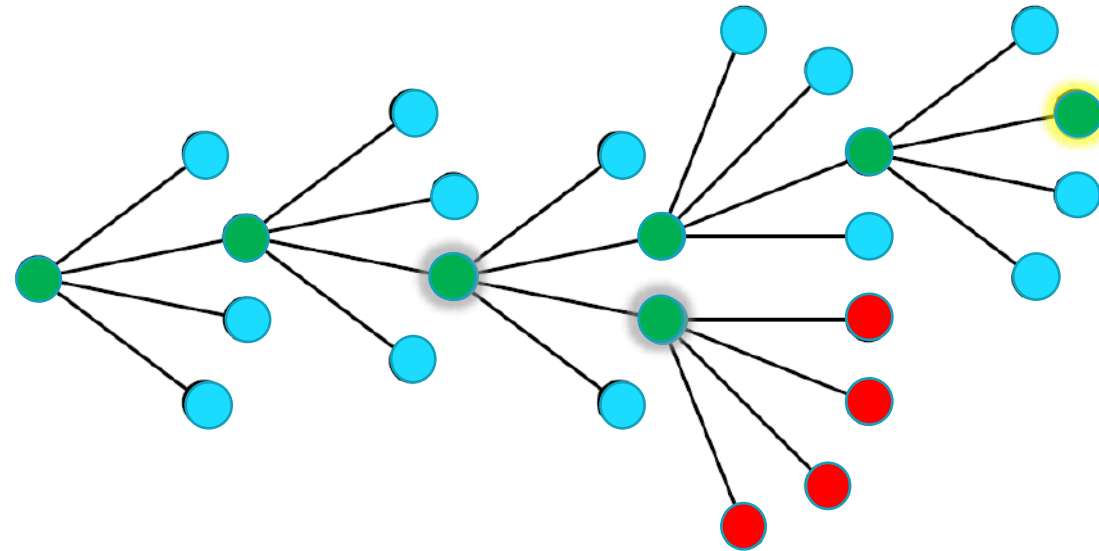
BACKTRACKING

- Constrói e percorre uma árvore de sub-tarefas.



BACKTRACKING

- Constrói e percorre uma árvore de sub-tarefas.



- A árvore de soluções cresce rapidamente (eventualmente com comportamento exponencial)

BACKTRACKING

- Aplicado a problemas que possuem *soluções parciais candidatas*:
 - Soluções gradativas, avaliadas a cada passo;
 - Teste rápido para determinar se uma solução parcial é candidata a uma solução válida.
- Utiliza-se uma estrutura de dados de controle.
 - Árvore, pilha...

BACKTRACKING

- Aplicações:
 - Satisfação de restrições (Palavras cruzadas, sudoku);
 - Interpretação de expressões ou textos (*parsing*);
 - Linguagens de programação lógicas;
 - Problemas combinatórios.

BACKTRACKING

- Percorre-se a árvore recursivamente (**profundidade**).
- Verifica-se cada elemento sobre **possível solução**:
 - Se **não é aceitável**, toda a subárvore é podada;
 - Se é **aceitável**, verifica-se se é uma solução completa;
 - Caso não seja, continua com um próximo candidato.
 - Caso seja, retorna.
- Recursivamente, inicia nova sequência com o conjunto de soluções candidatas;

BACKTRACKING – ALGORITMO GERAL

```
Função: Tenta(estado_atual){  
    // 1. Condição de Parada (Encontramos a solução?)  
    se solucaoDefinitiva(estado_atual) {  
        retornaSolucao(estado_atual);  
        retorna VERDADEIRO; // ou continua, se quiser buscar TODAS as soluções  
    }  
    // 2. Geração e iteração dos candidatos  
    candidatos = gerarCandidatos(estado_atual);  
    para cada candidato em candidatos {  
        // 3. Poda: O candidato é válido?  
        se solucaoAceitavel(candidato, estado_atual) {  
            registraCandidato(candidato) // Faz o movimento (Avança)  
            // 4. Chamada Recursiva: Tenta o próximo passo a partir daqui  
            se Tenta(estado_atual_atualizado) == VERDADEIRO {  
                retorna VERDADEIRO; // Propaga o sucesso para cima  
            }  
            // 5. O BACKTRACKING: Deu errado lá na frente!  
            apagaRegistroAnterior(candidato); //Desfaz o movimento (Retrocede)  
        }  
    }  
    // Se testou todos os candidatos e nenhum funcionou, é um beco sem saída  
    retorna FALSO;  
}
```

BACKTRACKING – ALGORITMO GERAL

```
Função: Tenta(estado_atual){  
  // 1. Condição de Parada (Encontramos a solução?)  
  se solucaoDefinitiva(estado_atual) {  
    retornaSolucao(estado_atual);  
    retorna VERDADEIRO; // ou continua, se quiser buscar TODAS as soluções  
  }
```

- **Condição de Parada (solucaoDefinitiva):** É a primeira coisa que perguntamos ao entrar na função.
- "Já chegamos na saída do labirinto?".
 - Se sim, comemoramos, salvamos a resposta e encerramos.

BACKTRACKING – ALGORITMO GERAL

```
// 2. Geração e iteração dos candidatos  
candidatos = gerarCandidatos(estado_atual);  
para cada candidato em candidatos {
```

- **Laço de Opções (para cada candidato...):**
 - Se ainda não resolvemos o problema, olhamos para frente.
 - "Quais são as minhas opções agora?".
 - Em um labirinto, seria "posso ir para frente, direita ou esquerda".
 - No Sudoku, "posso testar os números de 1 a 9".

BACKTRACKING – ALGORITMO GERAL

```
// 3. Poda: O candidato é válido?  
se solucaoAceitavel(candidato, estado_atual) {
```

- **A Poda (**solucaoAceitavel**):**
 - Aqui entra a inteligência do algoritmo.
 - Antes de dar o passo, ele olha e diz: "Tem uma parede na direita, então a opção 'direita' não é aceitável".
 - Ele nem tenta ir por ali, economizando tempo.

BACKTRACKING – ALGORITMO GERAL

- **Ação e a Recursão** (`registraCandidato` e `Tenta()`):
- O passo é válido!
 - Então nós damos esse passo (`registraCandidato`) e chamamos a função `Tenta()` novamente.
 - É como se disséssemos: "Beleza, dei um passo. E agora, a partir desse novo lugar, o que eu faço?".
 - **Incremento da solução**

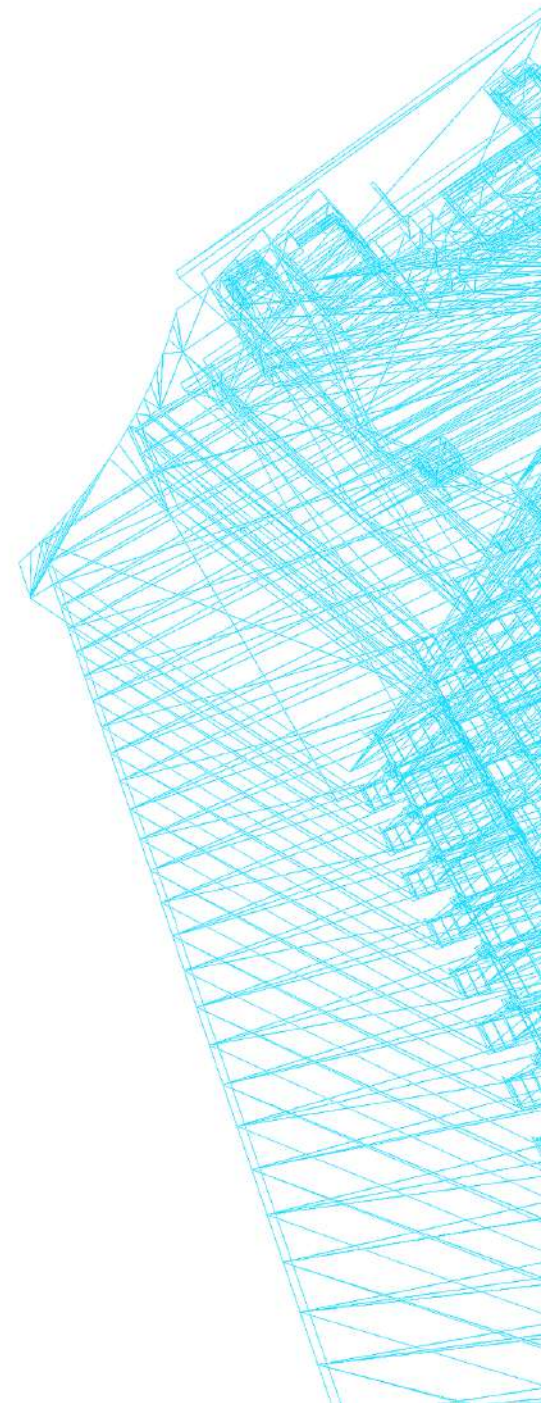
```
registraCandidato(candidato) // Faz o movimento (Avança)
// 4. Chamada Recursiva: Tenta o próximo passo a partir daqui
se Tenta(estado_atual_atualizado) == VERDADEIRO {
    retorna VERDADEIRO; // Propaga o sucesso para cima
}
```

BACKTRACKING – ALGORITMO GERAL

- **Coração do Backtracking** (`apagaRegistroAnterior`):
- Esse é o pulo do gato.
- Se a chamada recursiva `Tenta()` retornar **FALSO**, significa que o caminho que escolhemos lá no passo 4 deu num beco sem saída.
- O que fazemos?
 - **Desfazemos o passo** (`apagaRegistroAnterior`), limpamos a variável e o laço para naturalmente vai testar o *próximo* candidato disponível.

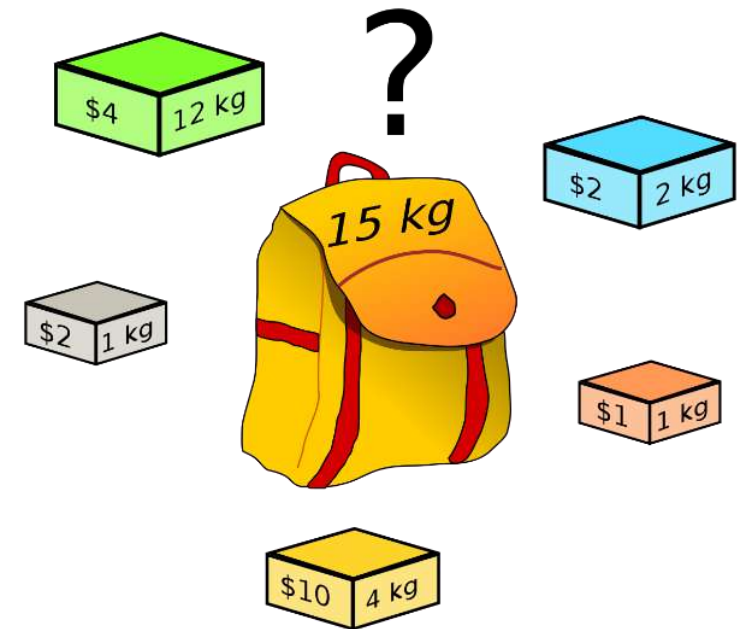
```
// 5. O BACKTRACKING: Deu errado lá na frente!  
apagaRegistroAnterior(candidato); //Desfaz o movimento (Retrocede)  
}  
}  
// Se testou todos os candidatos e nenhum funcionou, é um beco sem saída  
retorna FALSO;  
}
```

O PROBLEMA DA MOCHILA



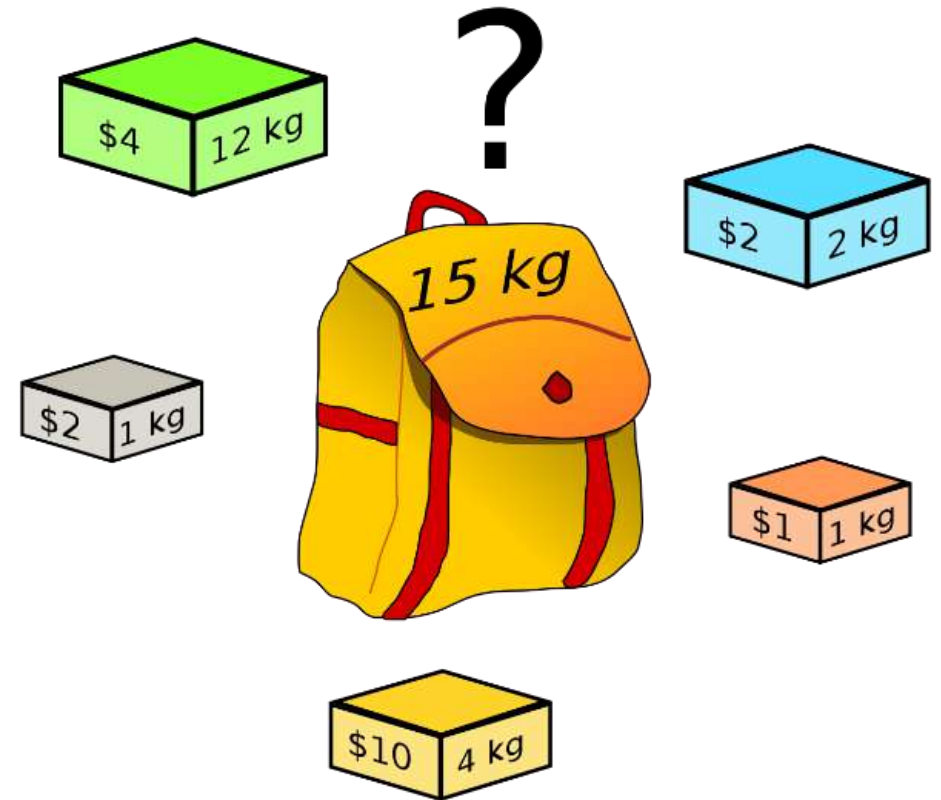
O PROBLEMA DA MOCHILA (KNAPSACK PROBLEM)

- Dados n itens, cada um com:
 - Pesos: $p_1, p_2, \dots, p_n$;
 - Valores: $v_1, v_2, \dots, v_n$;
- E uma mochila de capacidade C .
- Problema:
 - Encontrar o **subconjunto mais valioso** de itens que caibam dentro da mochila.



A MOCHILA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2



MOCHILA: ÁRVORE DE COMBINAÇÕES

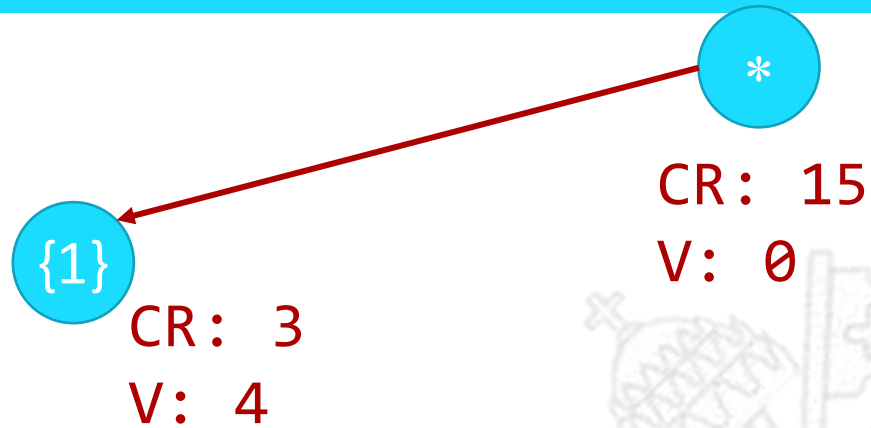
*

CR: 15

V: 0

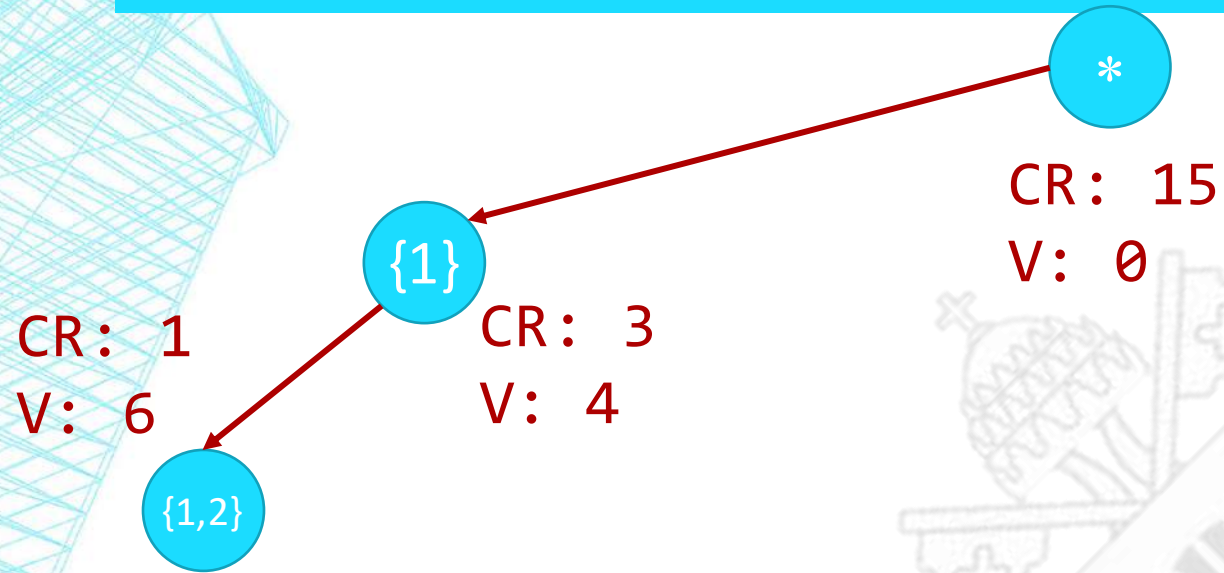
Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: ÁRVORE DE COMBINAÇÕES



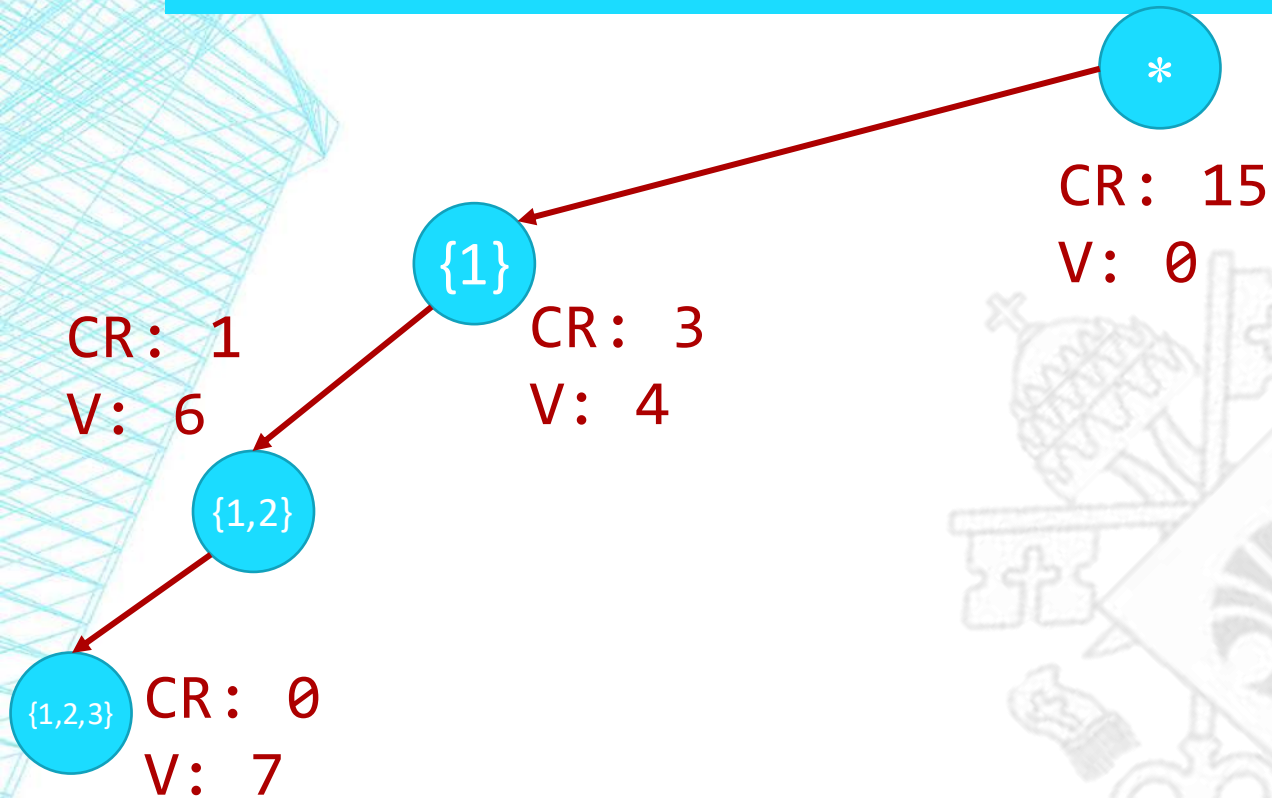
Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: ÁRVORE DE COMBINAÇÕES



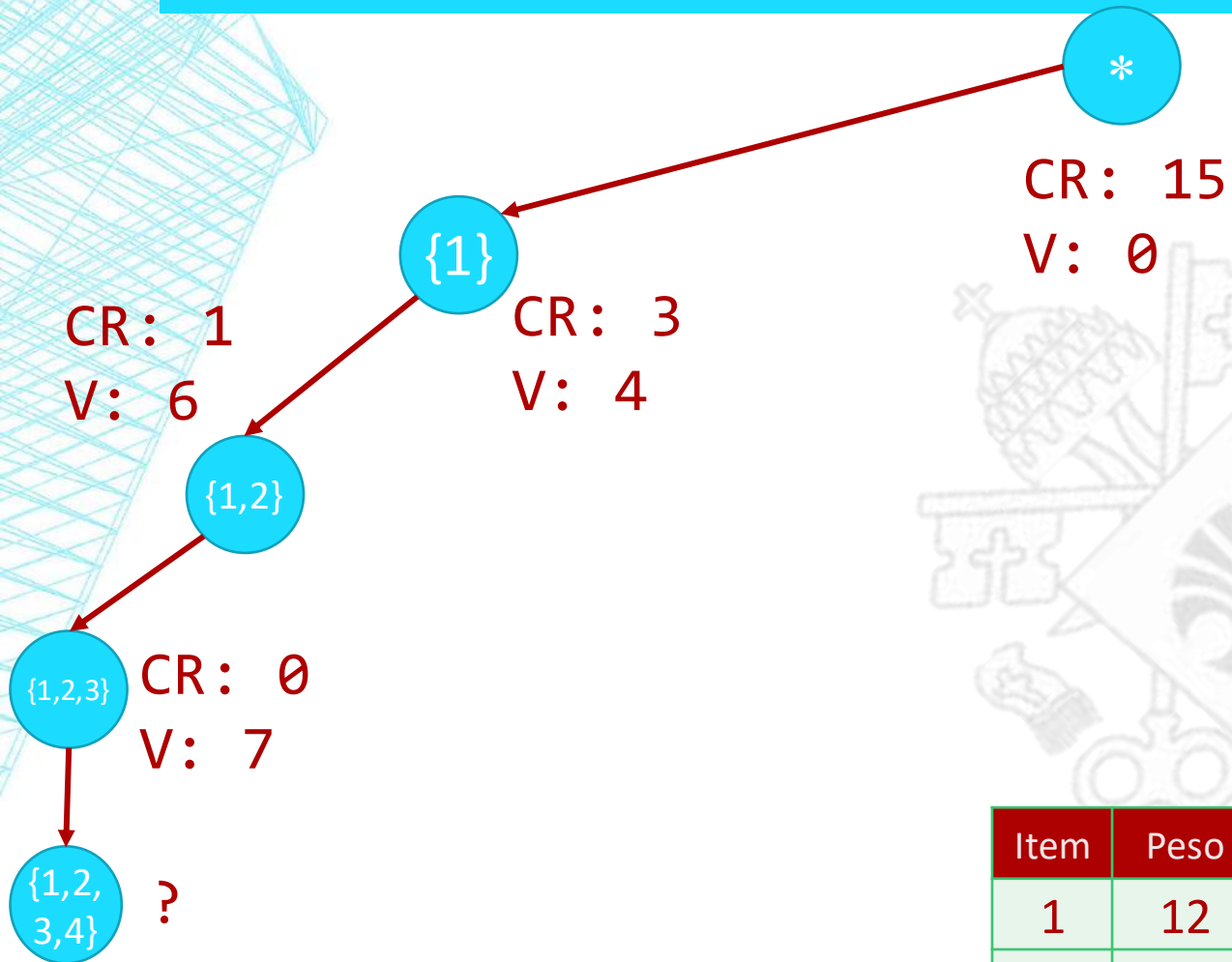
Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: ÁRVORE DE COMBINAÇÕES



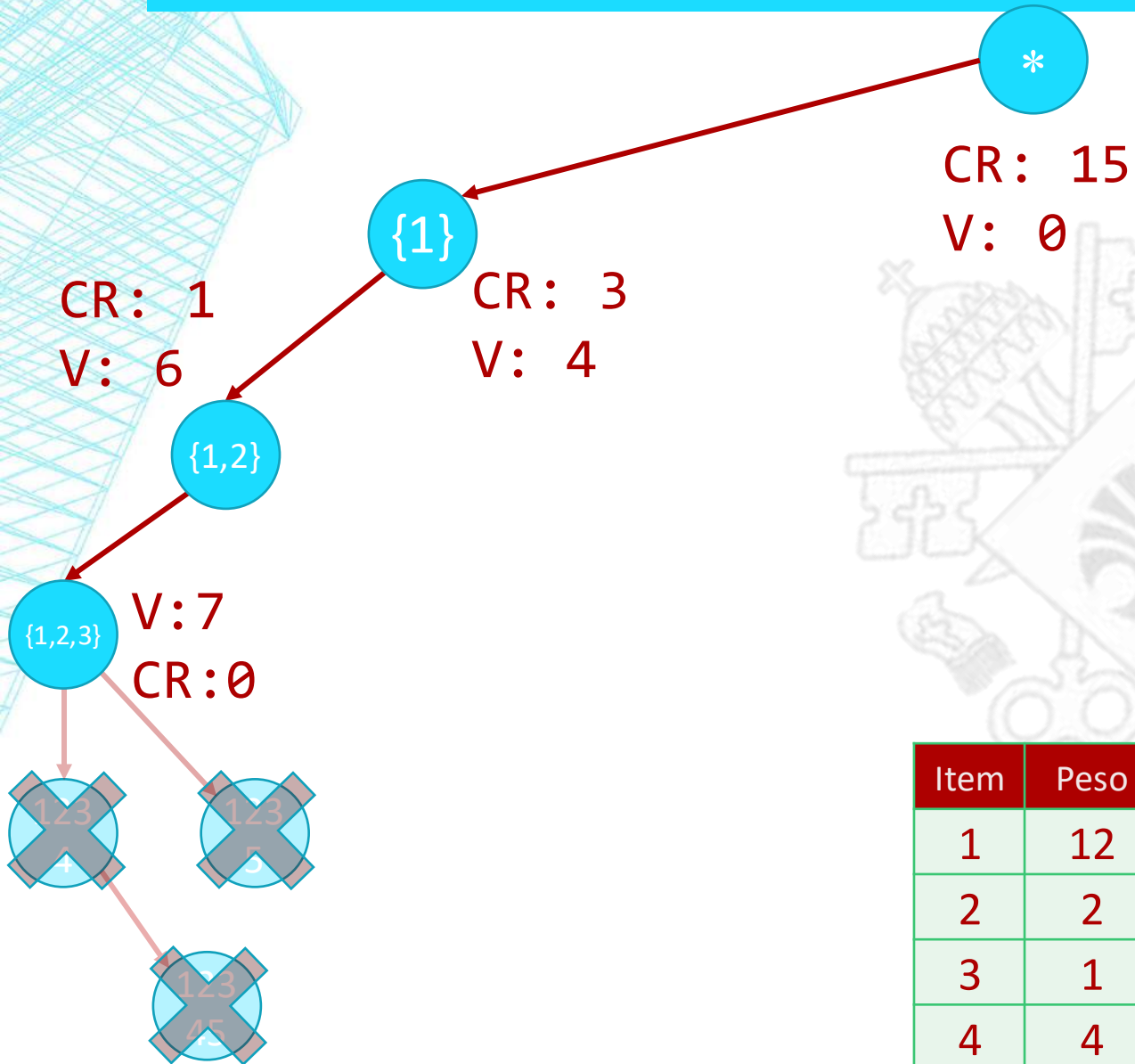
Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: ÁRVORE DE COMBINAÇÕES



Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

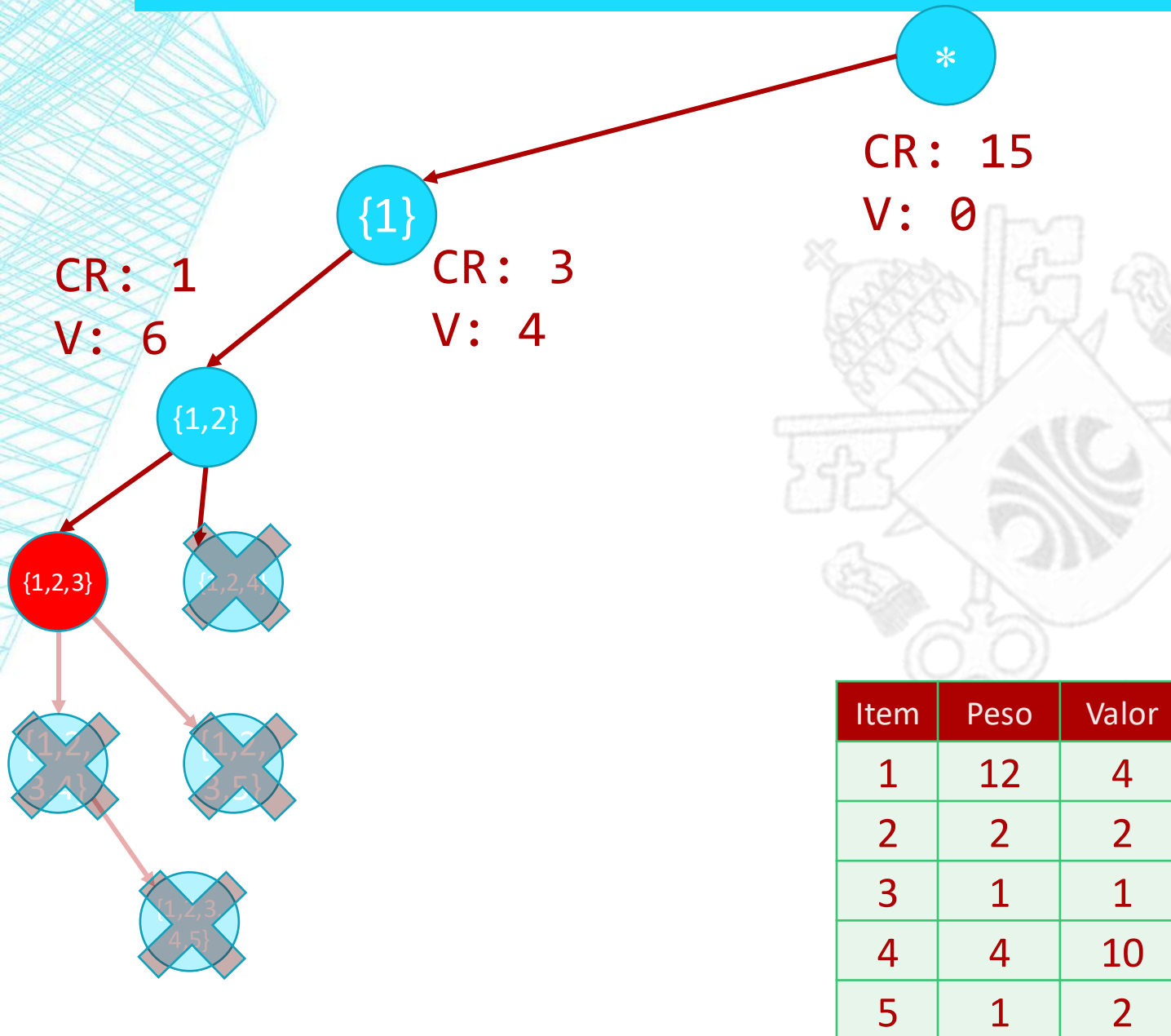
MOCHILA: ÁRVORE DE COMBINAÇÕES



Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

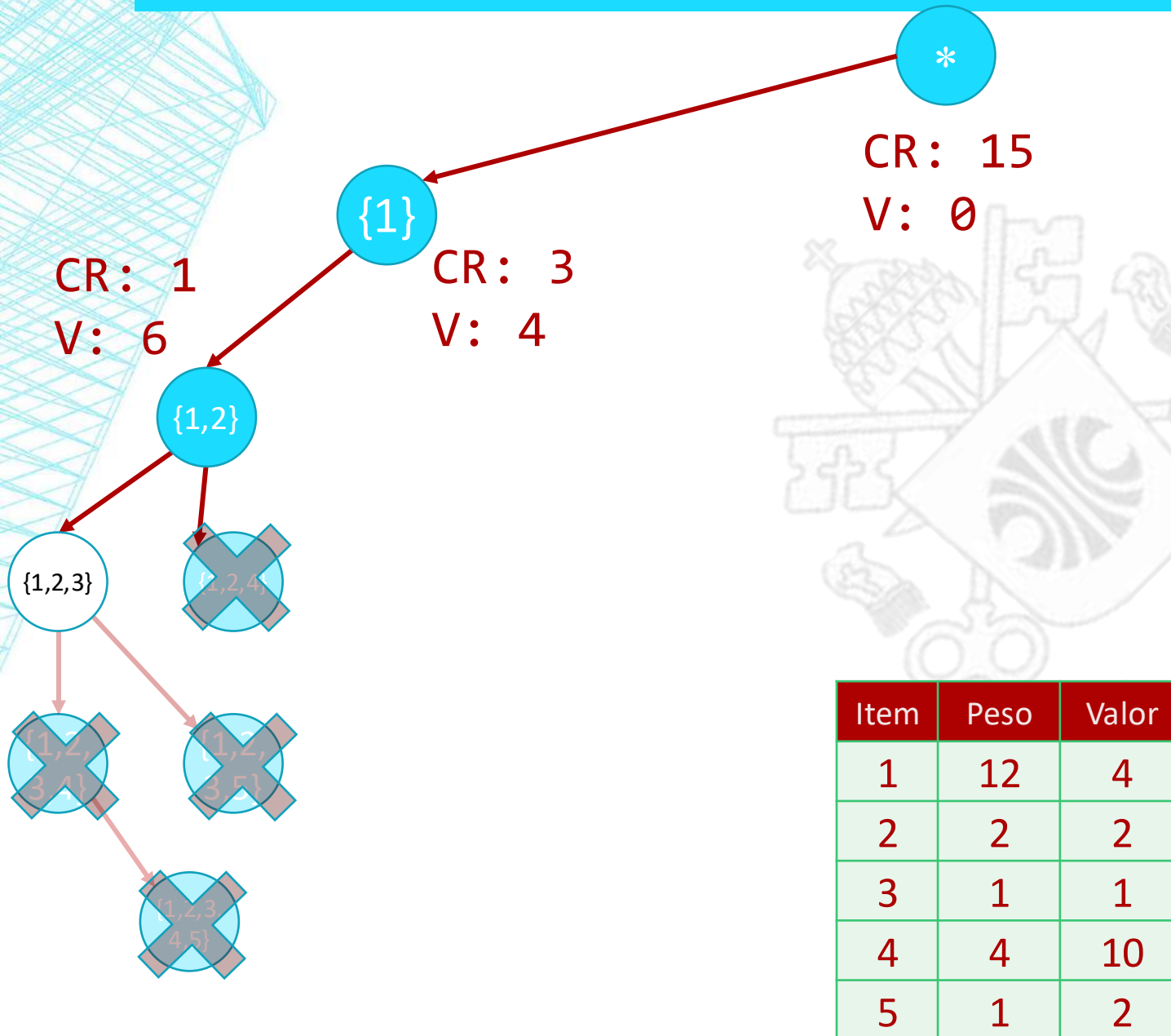
MOCHILA: ÁRVORE DE COMBINAÇÕES

Me1hor: 7



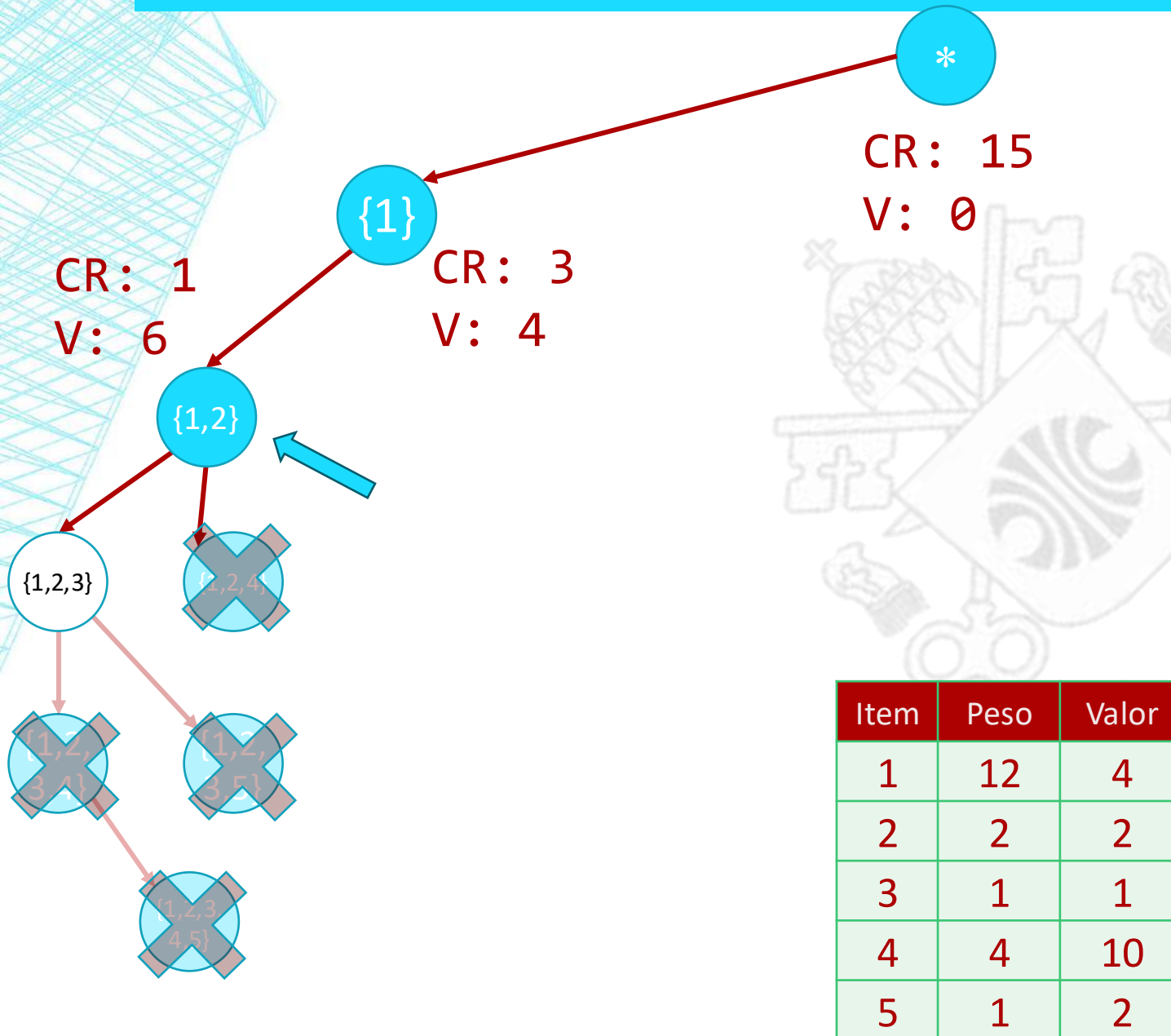
MOCHILA: ÁRVORE DE COMBINAÇÕES

Me1hor: 7



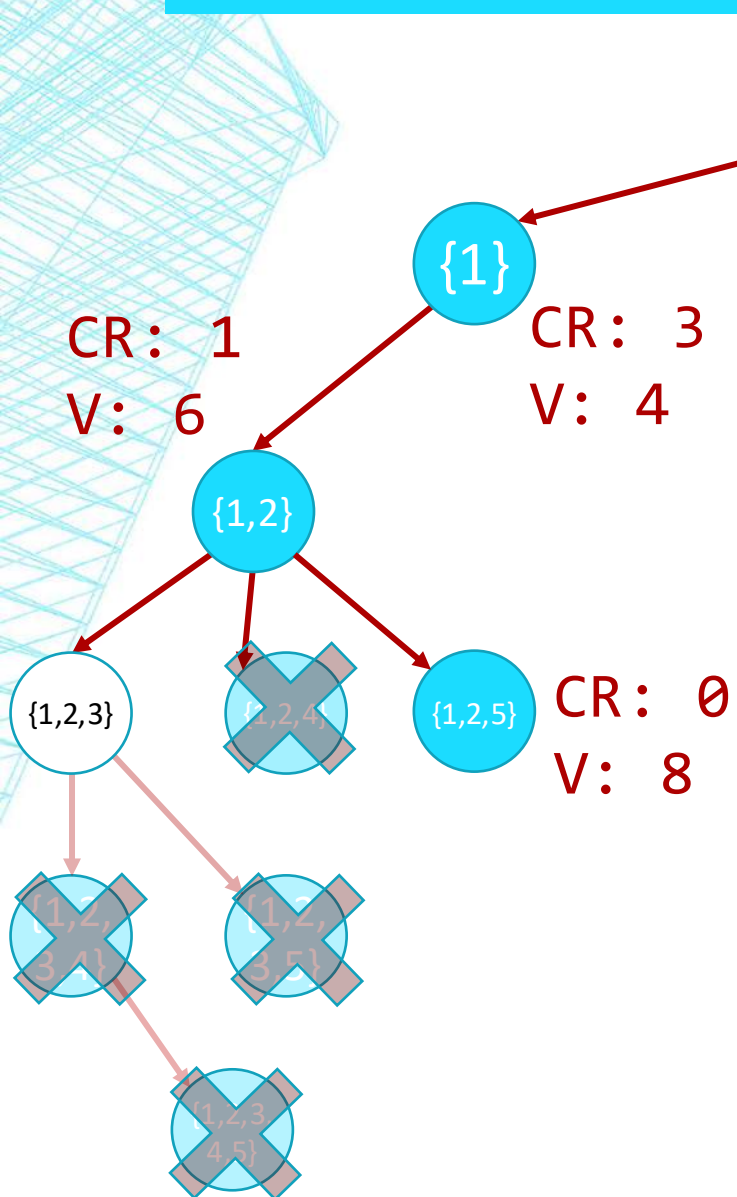
MOCHILA: ÁRVORE DE COMBINAÇÕES

Me1hor: 7



MOCHILA: ÁRVORE DE COMBINAÇÕES

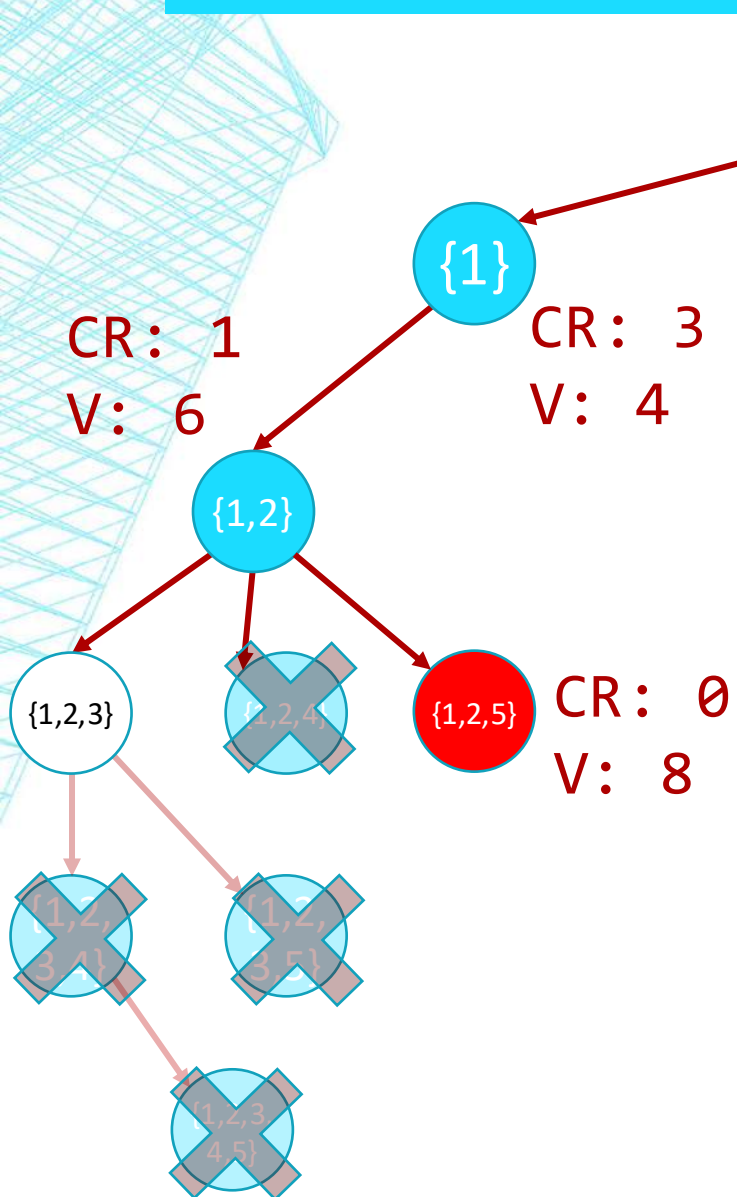
Me1hor: 7



Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: ÁRVORE DE COMBINAÇÕES

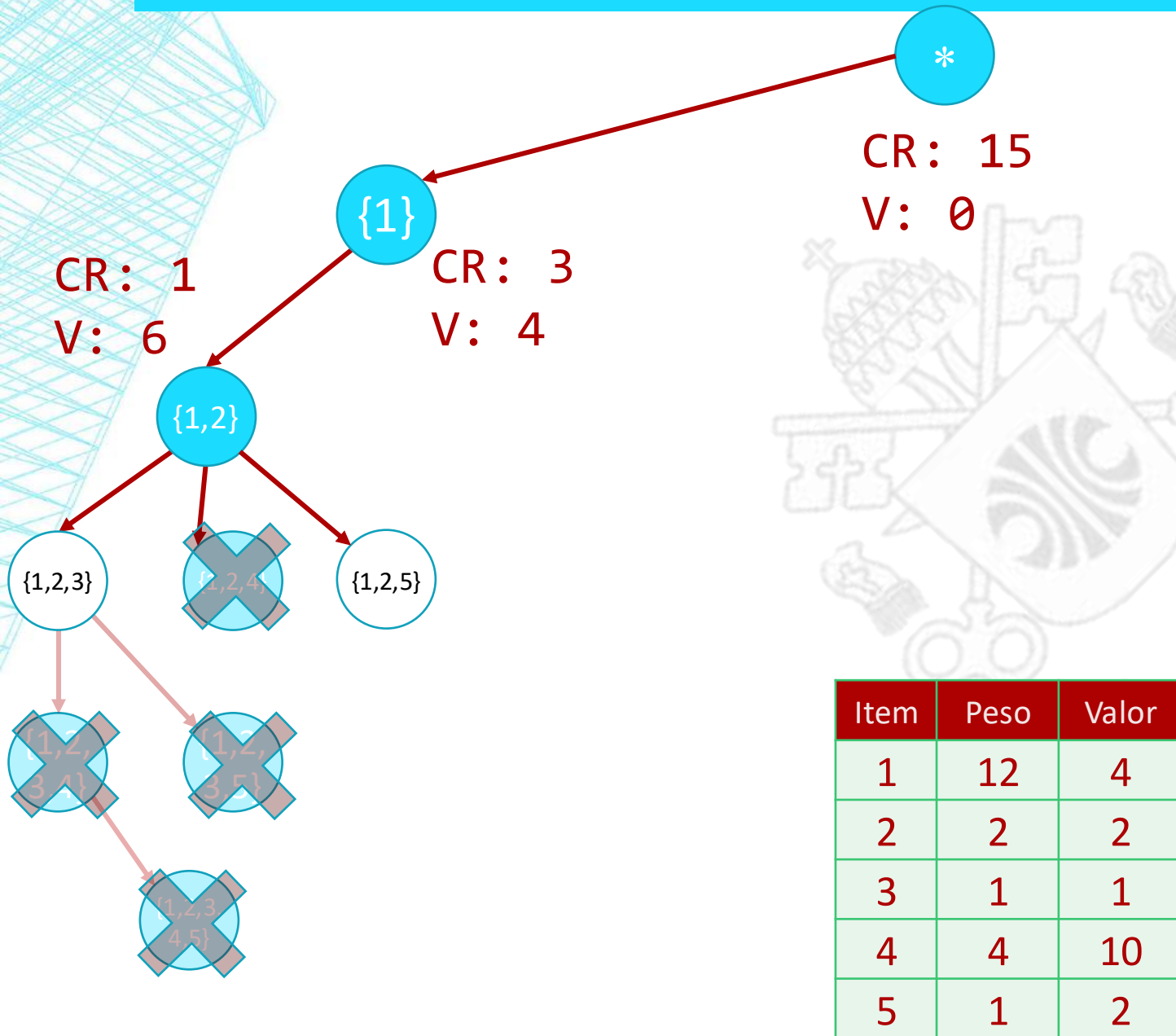
Me1hor: 8



Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

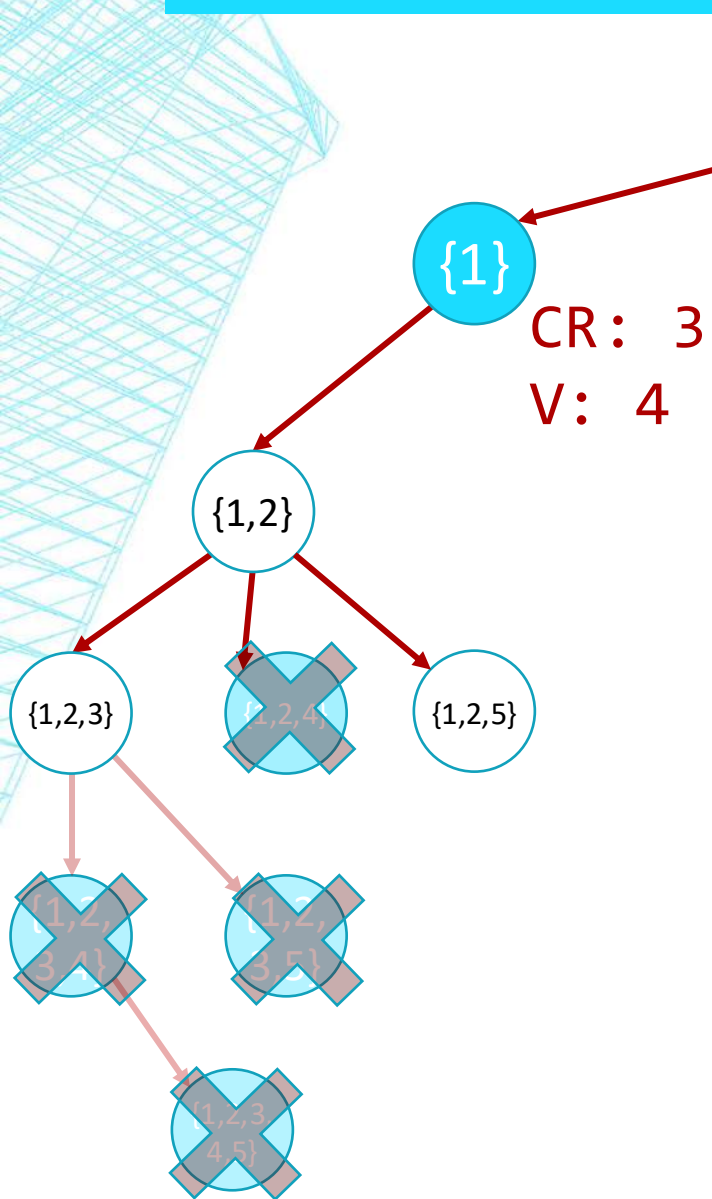
MOCHILA: ÁRVORE DE COMBINAÇÕES

Me1hor: 8



MOCHILA: ÁRVORE DE COMBINAÇÕES

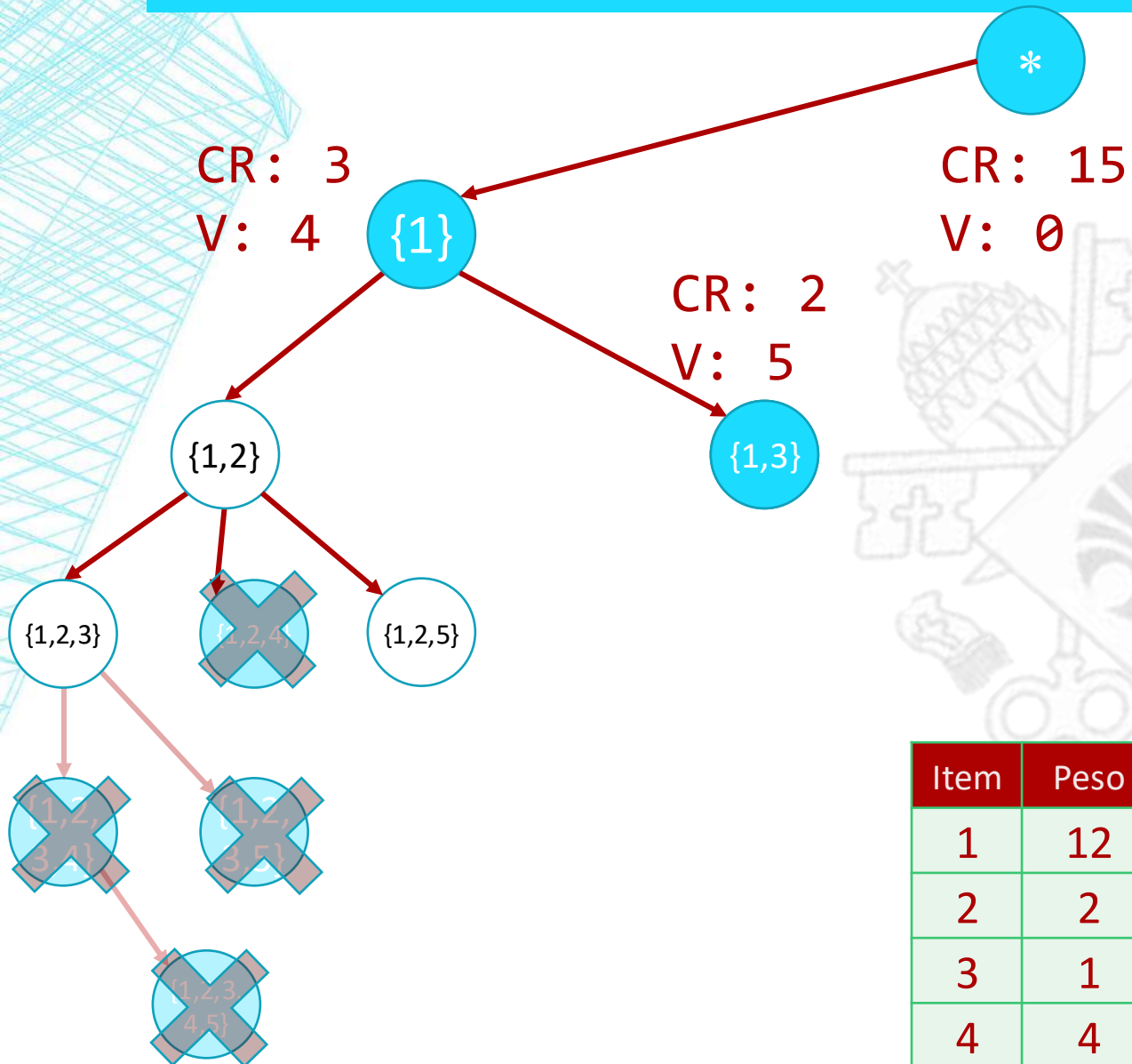
Me1hor: 8



Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: ÁRVORE DE COMBINAÇÕES

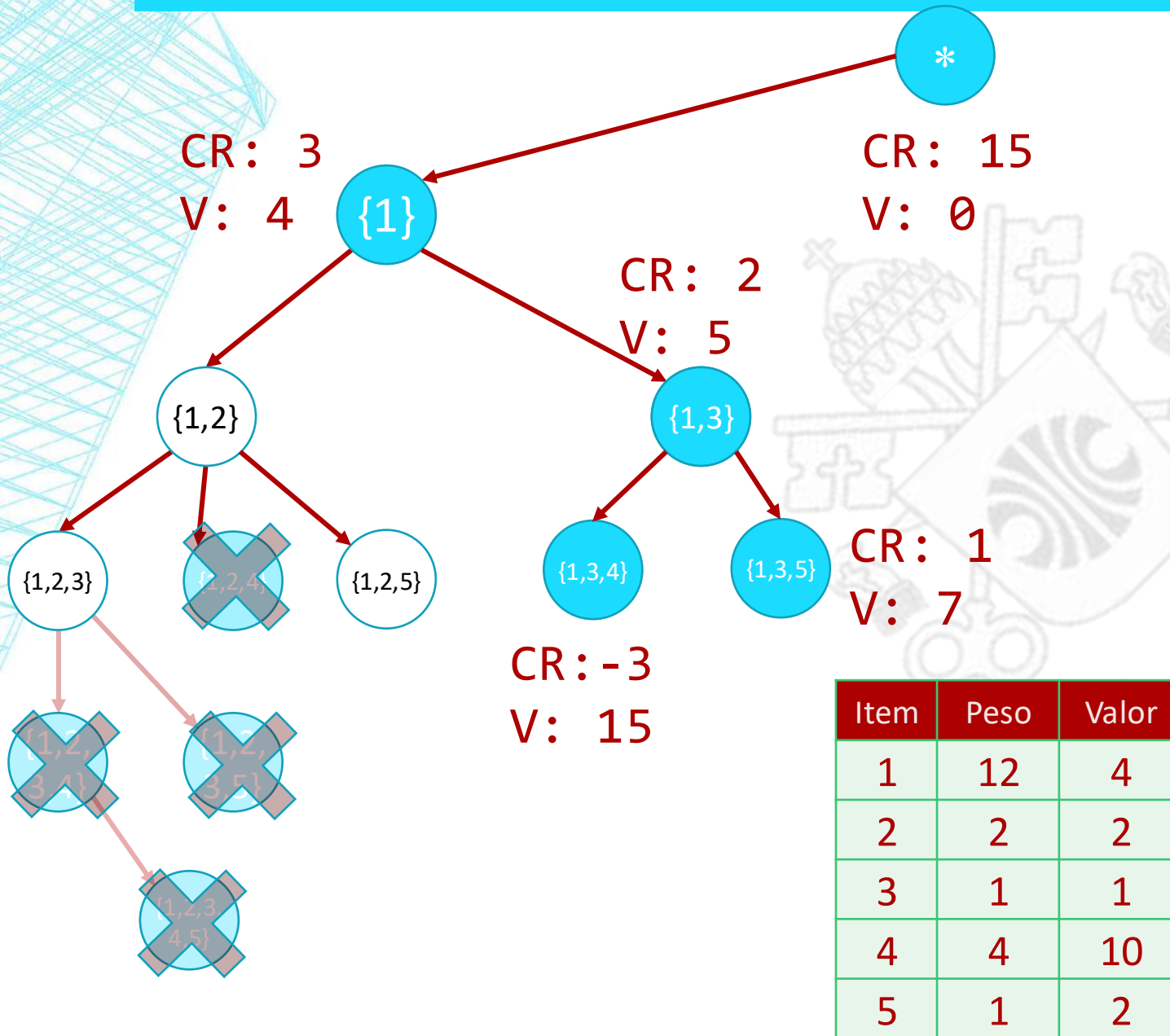
Me1hor: 8



Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

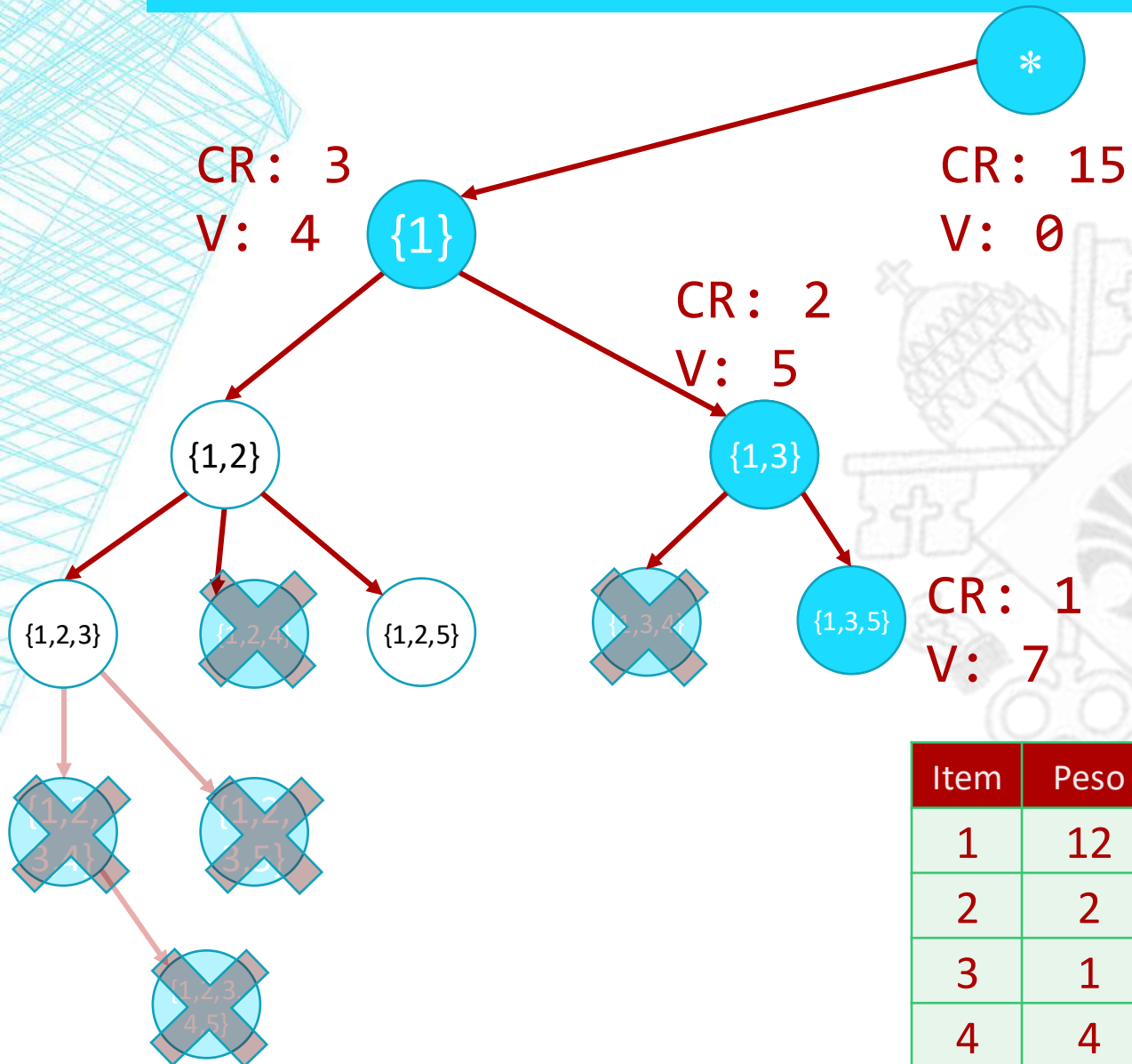
MOCHILA: ÁRVORE DE COMBINAÇÕES

MeLhor: 8



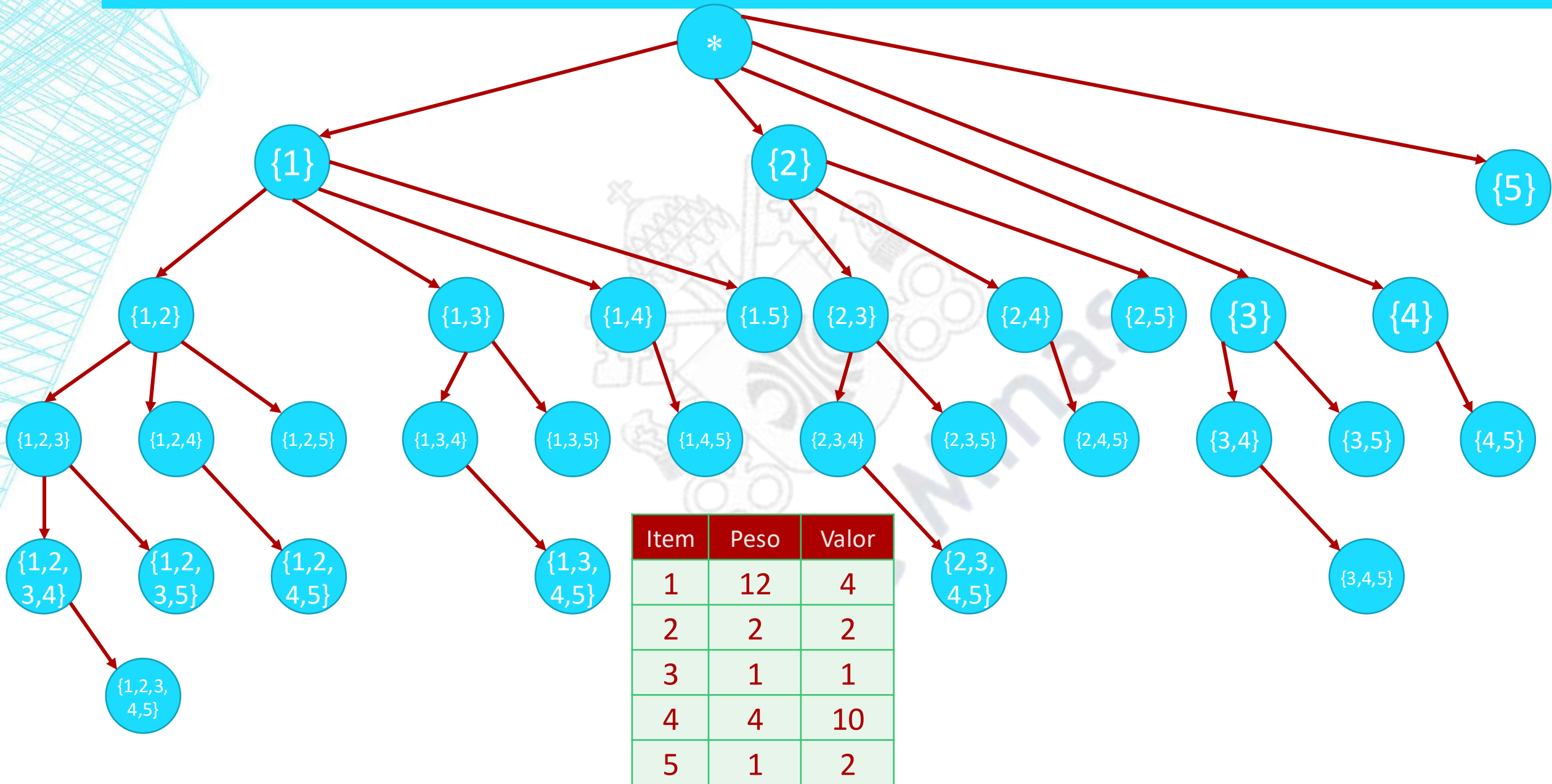
MOCHILA: ÁRVORE DE COMBINAÇÕES

MeLhor: 8

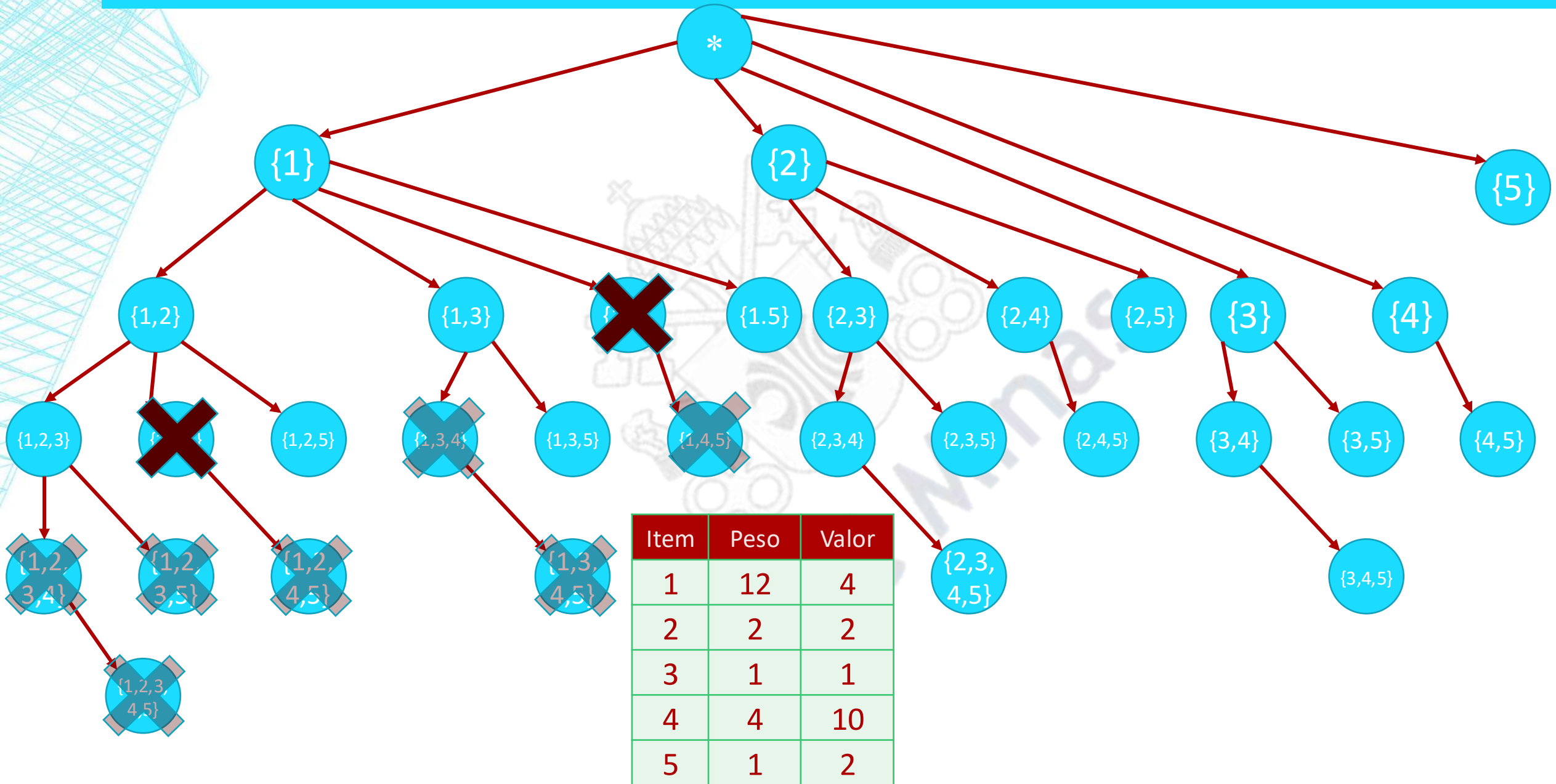


Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

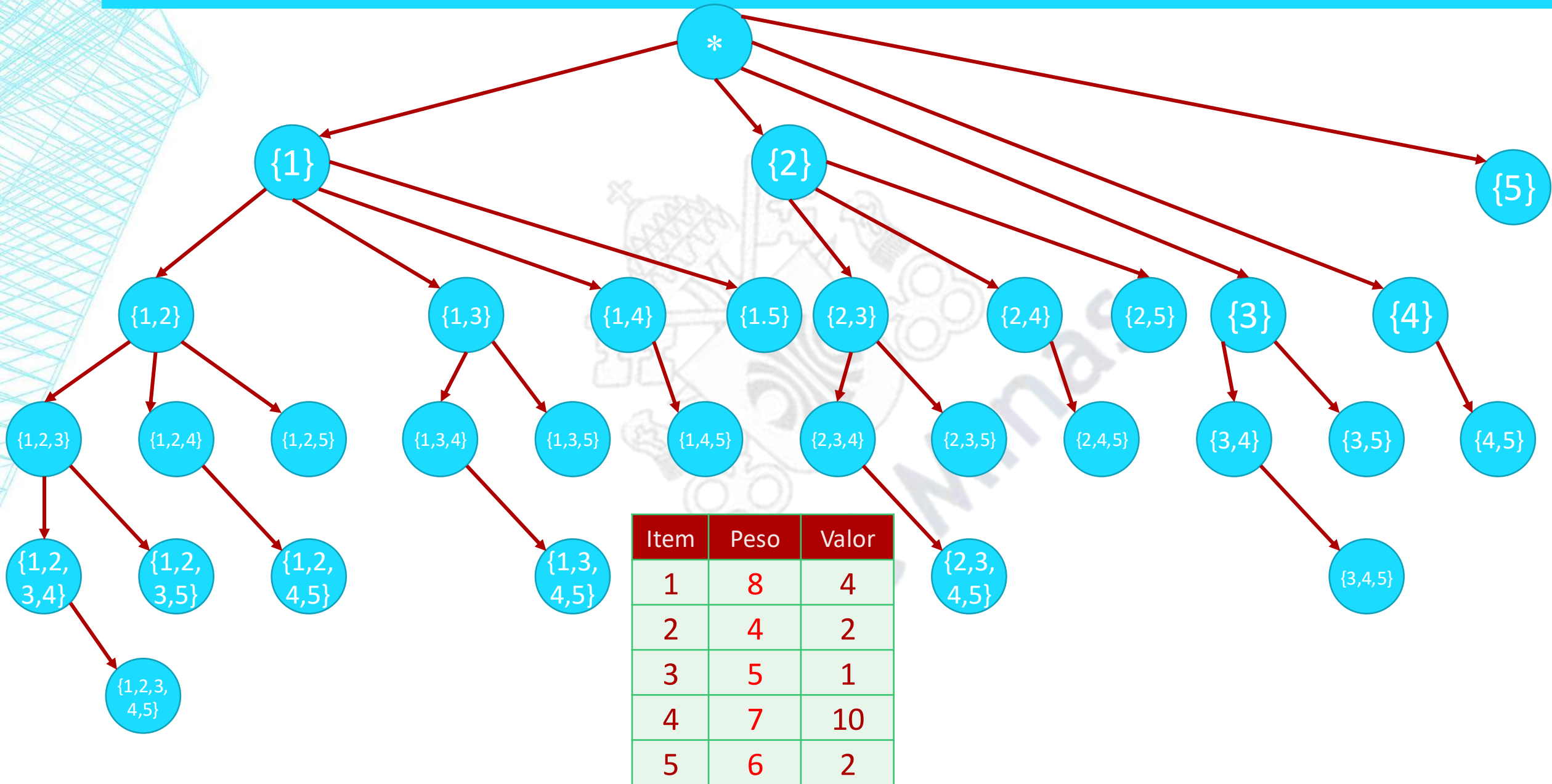
MOCHILA: ÁRVORE DE COMBINAÇÕES



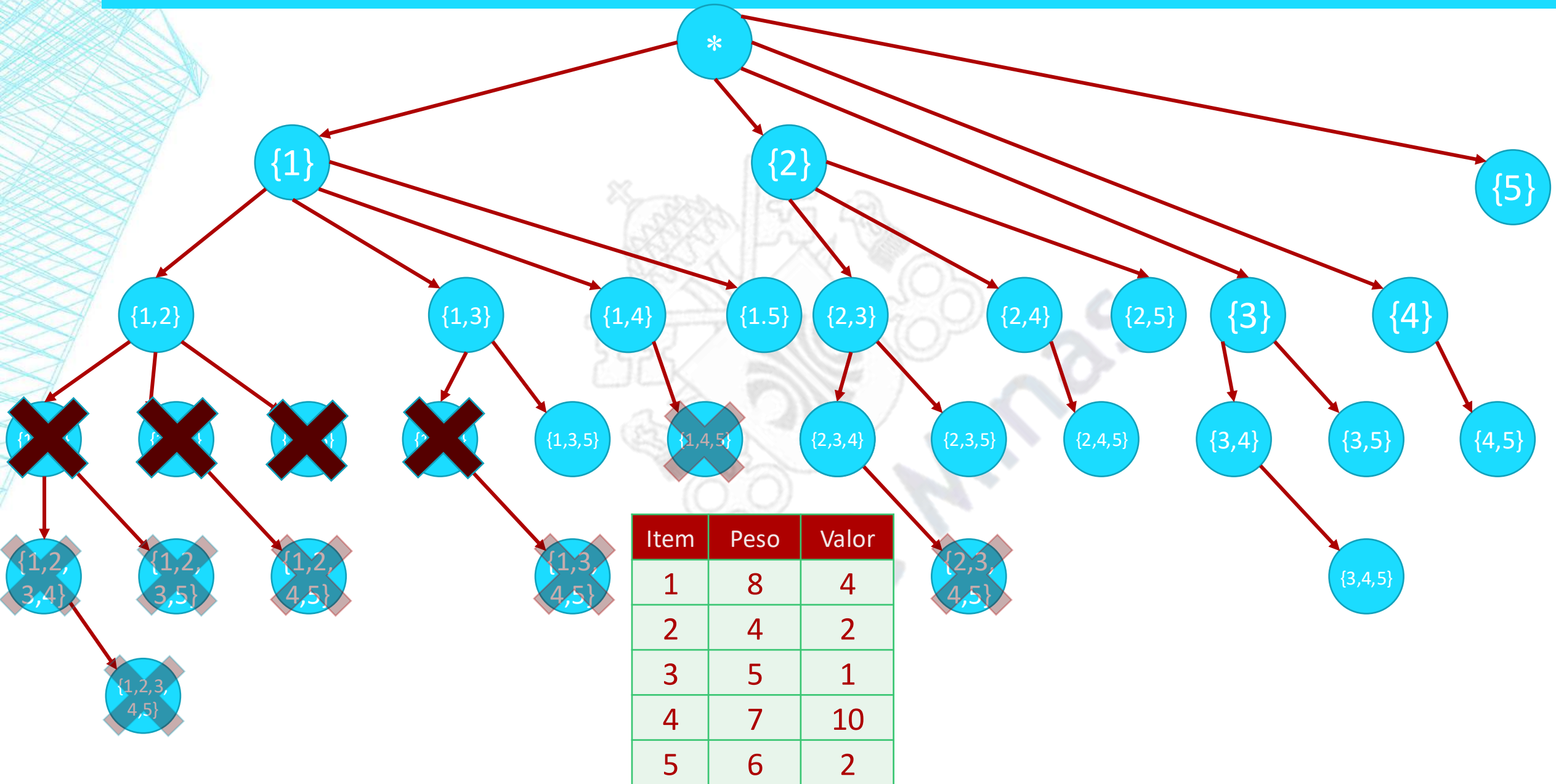
MOCHILA: ÁRVORE DE COMBINAÇÕES



MOCHILA: ÁRVORE DE COMBINAÇÕES



MOCHILA: ÁRVORE DE COMBINAÇÕES



PARA PENSAR: LEILÃO DE ENERGIA

- **Disponibilidade:** A empresa possui um total de X Megawatts (MW) para vender.
- **Os Lances:** Cada cliente oferece um valor V por um lote exato de K MW.
- **A Regra:** É "tudo ou nada". O cliente só compra se levar a quantidade exata que pediu.
- **O Objetivo:** Escolher a combinação de clientes que **maximize o lucro** sem ultrapassar a energia total (X).

PARA PENSAR: LEILÃO DE ENERGIA

- Se fôssemos resolver isso com Backtracking, quem é o nosso "estado atual"?
- Qual é a nossa **condição de parada**?
- O que tornaria um lance "inaceitável", obrigando o algoritmo a fazer a **poda** (voltar atrás)?